

RMMM Plan -- IBMS (Integrated Business Management Suite)

Field	Detail
Project Name	Integrated Business Management Suite (IBMS) v2.0
Team	Shivansh Srivastava (Product Developer), Prahallad Padhan (Product Owner), Ranveer Rai Khare (Scrum Master)
Sprint	Sprint 1
Date	April 14, 2026
Document Type	Risk Mitigation, Monitoring, and Management (RMMM) Plan

1. Introduction

This document defines the Risk Mitigation, Monitoring, and Management (RMMM) plan for the IBMS platform. IBMS is an AI-first ERP system built on FastAPI, Frappe, MongoDB, MariaDB, and Redis, deployed via Docker, Kubernetes, and cloud infrastructure (AWS/GCP). The RMMM plan identifies project and product risks, establishes mitigation strategies, defines monitoring approaches, and outlines management contingency actions.

2. Risk Identification

2.1 Risk Categories

ID	Category	Description
CAT-01	Technical	Risks related to architecture, code quality, and dependencies

ID	Category	Description
CAT-02	Security	Risks related to authentication, authorization, data breaches
CAT-03	Performance	Risks related to system scalability, response times, resources
CAT-04	Data Integrity	Risks related to data consistency across dual databases
CAT-05	Infrastructure	Risks related to deployment, cloud services, availability
CAT-06	Integration	Risks related to third-party services, webhooks, APIs
CAT-07	Schedule/Resource	Risks related to project timeline, team capacity
CAT-08	AI/ML Model	Risks related to model accuracy, bias, and reliability

2.2 Risk Register

Risk ID	Category	Risk Description	Probability	Impact	Exposure (P×I)	Priority
R-001	CAT-01	Dual database architecture (MongoDB + MariaDB) increases complexity and data sync failures	Medium	High	High	Critical
R-002	CAT-02	JWT secret key compromise leading to unauthorized access to all API endpoints	Low	Critical	High	Critical
R-003	CAT-02	Brute-force attack on /api/auth/login endpoint bypassing rate limiter	Medium	High	High	Critical
R-004	CAT-03	MongoDB collection size growth in kpi_snapshots and audit_logs degrades query performance	High	Medium	High	High
R-005	CAT-03	WebSocket /ws/kpi connection storms under high concurrent user load	Medium	Medium	Medium	High
R-006	CAT-04	Data inconsistency between MariaDB ERP entities and MongoDB analytics collections	Medium	High	High	Critical

Risk ID	Category	Risk Description	Probability	Impact	Exposure (P×I)	Priority
R-007	CAT-05	Redis service failure causing cascading degradation in caching and rate limiting	Medium	Medium	Medium	High
R-008	CAT-05	Kubernetes pod eviction under memory pressure (limit: 4Gi per pod)	Low	High	Medium	Medium
R-009	CAT-06	Webhook delivery failures to external systems due to timeout or endpoint unavailability	High	Low	Medium	Medium
R-010	CAT-06	Frappe framework version incompatibility after upstream updates	Medium	Medium	Medium	Medium
R-011	CAT-07	Single developer dependency -- key person risk for core module knowledge	High	High	High	Critical
R-012	CAT-08	AI anomaly detection (IsolationForest) producing false positives flagging legitimate transactions	High	Medium	High	High
R-013	CAT-08	AI Copilot returning inaccurate or misleading business insights	Medium	High	High	High
R-014	CAT-02	TOTP 2FA secret exposure through unprotected backup or log leak	Low	Critical	High	Critical
R-015	CAT-05	Docker Compose production deployment lacking proper secret management	Medium	High	High	High
R-016	CAT-01	In-memory fallback stores losing data on process restart when Redis/MongoDB unavailable	Medium	Medium	Medium	Medium
R-017	CAT-03	Background KPI refresh scheduler (15s interval) consuming excessive CPU at scale	Medium	Medium	Medium	Medium
R-018	CAT-04	MariaDB and MongoDB schema drift over application versions without migration tooling	Medium	High	High	High
R-019	CAT-05	GCP Cloud Run cold start latency impacting first-request user experience	High	Low	Medium	Low
R-020	CAT-02	CORS misconfiguration in production allowing unauthorized origins	Low	High	Medium	Medium

3. Risk Mitigation Strategies

3.1 Technical Risks

Risk ID	Mitigation Strategy	Owner	Timeline
R-001	Implement a database transaction coordinator service that ensures atomic operations across MongoDB and MariaDB. Add idempotency keys to all cross-database writes. Establish clear data ownership boundaries -- ERP CRUD in MariaDB, analytics/AI in MongoDB.	Developer	Sprint 2
R-010	Pin Frappe framework version in requirements.txt. Run compatibility tests in CI before upgrading. Maintain a Frappe API abstraction layer (conftest.py mock pattern).	Developer	Ongoing
R-016	Add persistent backup write-ahead log for in-memory fallback stores. Implement automatic resync when Redis/MongoDB recovers. Alert on fallback mode activation.	Developer	Sprint 3

3.2 Security Risks

Risk ID	Mitigation Strategy	Owner	Timeline
R-002	Rotate JWT secrets on a scheduled basis (90-day rotation policy). Store secrets in cloud-native secret managers (AWS Secrets Manager / GCP Secret Manager). Enforce minimum 256-bit key length. Never log JWT secrets.	Developer / DevOps	Sprint 2
R-003	Current rate limiter (IP + user-based) is in place. Enhance with progressive delay on consecutive failures. Add CAPTCHA after 5 failed attempts. Integrate fail2ban or cloud WAF for persistent attackers.	Developer	Sprint 2

Risk ID	Mitigation Strategy	Owner	Timeline
R-014	Encrypt TOTP secrets at rest in MongoDB using AES-256. Exclude TOTP fields from all API responses and log outputs. Audit access to 2FA configuration endpoints.	Developer	Sprint 2
R-015	Migrate Docker Compose production secrets to Docker Secrets or external vaults. Validate with <code>?required</code> annotations (already in <code>docker-compose.prod.yml</code>). Remove all hardcoded defaults from env vars.	DevOps	Sprint 2
R-020	Restrict CORS <code>allow_origins</code> to explicit production domains. Disable wildcard origins in production. Audit CORS headers in CI pipeline.	Developer	Sprint 2

3.3 Performance Risks

Risk ID	Mitigation Strategy	Owner	Timeline
R-004	Implement TTL indexes on <code>kpi_snapshots</code> (90-day retention) and <code>audit_logs</code> (365-day retention). Add compound indexes on (<code>metric_code</code> , <code>recorded_at</code>). Schedule nightly archival jobs.	Developer	Sprint 2
R-005	Implement WebSocket connection pooling with per-user limits (max 3 concurrent). Add backpressure via message queue buffering. Load test with 500+ concurrent WebSocket clients.	Developer	Sprint 3
R-017	Make KPI refresh interval configurable via environment variable. Implement adaptive scheduling that adjusts frequency based on active user count. Add circuit breaker for KPI refresh when system is under load.	Developer	Sprint 3
R-019	Configure GCP Cloud Run minimum instances to 1 for production. Pre-warm critical paths. Optimize Docker image layers for faster cold start.	DevOps	Sprint 2

3.4 Data Integrity Risks

Risk ID	Mitigation Strategy	Owner	Timeline
---------	---------------------	-------	----------

Risk ID	Mitigation Strategy	Owner	Timeline
R-006	Implement event-driven synchronization -- MariaDB writes publish events that trigger MongoDB analytics updates. Add daily reconciliation job comparing record counts and checksums.	Developer	Sprint 3
R-018	Introduce database migration scripts for both MariaDB (Alembic) and MongoDB (mongomigrate). Version-lock schemas with migration checksums. Run migrations as part of CI/CD pipeline.	Developer	Sprint 2

3.5 Infrastructure Risks

Risk ID	Mitigation Strategy	Owner	Timeline
R-007	Redis graceful fallback is already implemented (in-memory dict). Add Redis Sentinel or cluster mode for production HA. Monitor Redis memory usage with alerts at 80% threshold.	DevOps	Sprint 3
R-008	Right-size Kubernetes resource requests based on profiling data. Implement Horizontal Pod Autoscaler (HPA) with CPU/memory targets. Set Pod Disruption Budgets (PDB) for minimum availability.	DevOps	Sprint 3

3.6 Integration Risks

Risk ID	Mitigation Strategy	Owner	Timeline
R-009	Implement exponential backoff retry (3 attempts) for webhook deliveries. Add dead letter queue for persistently failing webhooks. Dashboard alerting for webhook failure rate > 5%.	Developer	Sprint 2

3.7 Schedule/Resource Risks

Risk ID	Mitigation Strategy	Owner	Timeline
---------	---------------------	-------	----------

Risk ID	Mitigation Strategy	Owner	Timeline
R-011	Document all core modules with architecture decision records (ADRs). Establish code review requirements. Create runbooks for operational procedures. Cross-train team members on critical modules.	Team	Ongoing

3.8 AI/ML Risks

Risk ID	Mitigation Strategy	Owner	Timeline
R-012	Add configurable confidence thresholds for anomaly alerts (currently 0.75). Implement human-in-the-loop review for high-impact AI decisions. Track false positive rate as a KPI. Retrain models nightly (jobs/retrain_models.py).	Developer	Sprint 3
R-013	Add disclaimers to AI Copilot responses. Implement confidence scoring for all AI outputs. Log all Copilot interactions for review. Provide source references for insights (KPI snapshot data links).	Developer	Sprint 2

4. Risk Monitoring

4.1 Monitoring Matrix

Risk ID	Monitoring Mechanism	Metric/Indicator	Threshold	Frequency
R-001	Cross-database reconciliation job	Record count delta (MariaDB vs MongoDB)	Delta > 100 records	Daily (3 AM)
R-002	Audit log analysis (/api/audit/log)	Unusual token usage patterns, multi-location access	> 10 unusual events/hour	Real-time

Risk ID	Monitoring Mechanism	Metric/Indicator	Threshold	Frequency
R-003	Rate limiter logs + /api/metrics	Failed login attempts per IP	> 20 failures/minute	Real-time
R-004	MongoDB collection stats	kpi_snapshots document count, avg query time	> 10M docs or > 500ms query	Weekly
R-005	WebSocket connection counter (/api/health)	Active WS connections	> 1000 concurrent	Real-time
R-006	Reconciliation job	Hash mismatch between DB records	Any mismatch	Daily
R-007	/api/health Redis probe	Redis connectivity status	degraded state	Every 15s
R-008	Kubernetes metrics (kube-state-metrics)	Pod eviction events, OOMKilled count	Any eviction	Real-time
R-009	Webhook log analysis (Integration Webhook Log)	Webhook failure rate	> 5% failure rate	Hourly
R-011	Git commit analytics	Commits per module per author	Single-author module risk	Monthly
R-012	AI Alert tracking	False positive rate for anomaly alerts	> 30% FP rate	Weekly
R-013	Copilot interaction logs	User feedback / correction rate	> 20% corrections	Weekly
R-015	Security scan (CI pipeline)	Hardcoded secrets detection	Any finding	Per commit
R-017	/api/system/status	CPU usage during KPI refresh	> 80% CPU during refresh	Real-time

4.2 Existing Monitoring Infrastructure

The IBMS platform already includes the following monitoring capabilities:

Component	Endpoint/Module	Capabilities
-----------	-----------------	--------------

Component	Endpoint/Module	Capabilities
Health Check	GET /api/health	Redis, MongoDB, MariaDB connectivity; uptime; error rate; WS connection count
Metrics	GET /api/metrics	Request count, error count, last refresh timestamp
System Status	GET /api/system/status	Per-endpoint latency stats, comprehensive system health
Audit Trail	GET /api/audit/log	Tamper-evident audit logging with base64-encoded payloads
Frappe Health	monitoring/healthcheck.py	Database connection health (ok/degraded)
Request Tracing	monitoring/tracing.py	UUID-based trace IDs per request span
Request Metrics	monitoring/metrics.py	In-memory request counters with snapshot
K8s Probes	Readiness + Liveness	HTTP GET /api/method/ping every 15s

5. Risk Management (Contingency Plans)

5.1 Contingency Actions

Risk ID	Trigger Condition	Contingency Action	Responsible	Recovery Time Objective
R-001	Cross-database sync failure detected	1. Halt affected write operations. 2. Run manual reconciliation script. 3. Replay failed events from audit log. 4. Verify data integrity before resuming.	Developer	4 hours

Risk ID	Trigger Condition	Contingency Action	Responsible	Recovery Time Objective
R-002	JWT secret suspected compromised	1. Immediately rotate JWT secret in Secret Manager. 2. Invalidate all active tokens (clear refresh_tokens collection). 3. Force re-authentication for all users. 4. Audit recent access logs. 5. Investigate breach vector.	Developer + DevOps	1 hour
R-003	Sustained brute-force attack detected	1. Enable cloud WAF emergency rules. 2. Blocklist attacking IP ranges. 3. Temporarily increase rate limit strictness. 4. Notify security team.	DevOps	30 minutes
R-006	MariaDB-MongoDB data inconsistency confirmed	1. Identify affected records via reconciliation output. 2. Designate source of truth (MariaDB for ERP, MongoDB for analytics). 3. Re-derive analytics from ERP records. 4. Notify affected users of stale data window.	Developer	8 hours
R-007	Redis service fully down	1. System auto-falls back to in-memory stores (already implemented). 2. Alert operations team. 3. Restore Redis from snapshot backup. 4. Verify rate limiting and caching recovery.	DevOps	2 hours
R-008	K8s pods evicted due to resource pressure	1. Scale node pool (add nodes). 2. Review and adjust resource limits. 3. Apply HPA if not already active. 4. Investigate memory leak sources.	DevOps	1 hour

Risk ID	Trigger Condition	Contingency Action	Responsible	Recovery Time Objective
R-012	AI anomaly false positive rate exceeds 30%	1. Increase anomaly threshold to 0.85. 2. Disable auto-alerting temporarily. 3. Queue model retraining with updated training data. 4. Review and relabel flagged transactions.	Developer	24 hours
R-014	TOTP secret exposure detected	1. Invalidate all issued TOTP secrets. 2. Force 2FA re-enrollment for affected users. 3. Rotate encryption keys for at-rest TOTP storage. 4. Audit access patterns.	Developer	2 hours
R-015	Production secrets leaked	1. Rotate all affected secrets immediately. 2. Redeploy with new credentials. 3. Audit access logs for unauthorized usage. 4. Investigate leak source. 5. Revoke compromised cloud service keys.	DevOps	1 hour

5.2 Escalation Matrix

Severity Level	Description	Response Time	Escalation Path
SEV-1 (Critical)	System down, data breach, security compromise	15 minutes	Developer -> Scrum Master -> Product Owner
SEV-2 (High)	Major feature degraded, data inconsistency, high error rate	1 hour	Developer -> Scrum Master
SEV-3 (Medium)	Minor feature degraded, performance slowdown	4 hours	Developer (self-managed)

Severity Level	Description	Response Time	Escalation Path
SEV-4 (Low)	Cosmetic issues, minor bugs, improvement suggestions	Next sprint	Backlog prioritization

6. Risk Tracking Dashboard

6.1 Summary Metrics

Metric	Current Value	Target
Total Risks Identified	20	--
Critical Priority Risks	5	0 open by Sprint 4
High Priority Risks	7	<= 3 open by Sprint 4
Medium Priority Risks	7	Monitor continuously
Low Priority Risks	1	Accept
Risks with Mitigation in Place	12	20 (100%) by Sprint 3
Risks with Active Monitoring	14	20 (100%) by Sprint 3

6.2 Risk Heatmap

	Low Impact	Medium Impact	High Impact	Critical Impact
High Prob.		R-012	R-004, R-011	
Medium Prob.		R-005, R-007 R-016, R-017 R-010	R-001, R-003 R-006, R-013 R-015, R-018 R-020	R-002, R-014
Low Prob.			R-008	

High Prob.	R-009, R-019		
------------	--------------	--	--

7. Review Schedule

Activity	Frequency	Participants
Risk register review and update	Every sprint (bi-weekly)	Full team
Monitoring threshold calibration	Monthly	Developer + DevOps
Contingency plan drill (tabletop)	Quarterly	Full team
Full RMMM plan revision	Every 3 sprints	Product Owner + Developer
Post-incident risk register update	After every SEV-1/SEV-2	Full team

8. Approval

Role	Name	Date
Product Developer	Shivansh Srivastava	April 14, 2026
Product Owner	Prahallad Padhan	April 14, 2026
Scrum Master	Ranveer Rai Khare	April 14, 2026